GigBid: R&D Report

AI-4016 Assignment II · engineering design review

Ibrahim Umar Malik November 2026

How to read this report. Every number is rendered from a file in `evals/results/` by `python -m evals.make_tables` (CI fails if the report and the files disagree). Unless a table says otherwise, runs use: **real Upwork job posts, client signals and market prices** from six public Kaggle datasets, with **synthetic** freelancer portfolios, client reply behaviour and planted scams and injections (§2, `docs/DATA_CARD.md`); the `offline` model profile (deterministic heuristics in every LLM node, because this build environment had no model access); a hash embedder and a lexical reranker. The tune/test split is fixed (70/30 by row), and nothing was chosen on the test split. Where an experiment needs a real LLM it is marked **not run**, never estimated. `make eval-llm` reruns all of it with open-weight models.

1. Problem and constraints

A freelancer spends hours a week reading job posts, most of which are a poor fit, a scam or underpaid, and then writes proposals that either sound generic or overclaim. Connects cost money. GigBid's job: read every new post on a schedule, skip the bad ones with a reason, and for the good ones prepare a priced, evidence-backed proposal the freelancer can approve in under a minute.

Never automated: sending a proposal, sending a message, spending Connects, accepting a contract, and any action on Upwork outside its official API. GigBid prepares; the human sends. Live mode only creates drafts on Upwork.

Budgets (declared in `agent/config.yaml: budgets.declared`): p95 latency 25 s per job analysed and 60 s per proposal; USD 0.002 per job analysed and USD 0.01 per proposal on the open profile; at most 40 graph steps and 60k tokens per thread.

2. Data

The first build used fully synthetic jobs, because no data had been supplied. The version reported here runs on real posts. **Real:** 36,265 cleaned Upwork job posts with skill tags, budgets, Connects and client history (dataset A, Apache 2.0); paid rates from 1.29M freelance contracts (C, ODC-By); 2024 posted budgets (U, T, ODC-By). **Synthetic, and labelled so on every record:** the three freelancer portfolios (no public dataset has case studies, code and past proposals with outcomes), GigSim's client reply preferences (no public data links proposals to replies), and scam, injection and repost overlays on real posts (known ground truth for safety metrics). A sixth dataset (S) claimed to be freelancer statistics, but its median rate is $280/h for every skill. It turned out to be synthetic, so it was rejected for pricing. A fake-job corpus (F, licence unknown) is used only locally, as an external benchmark.

Data pipeline (data_pipeline/prepare.py)	Count
Upwork job posts (dataset A), raw	63,950
after de-duplication	59,594
after English / length filter	49,912
with a usable budget (the job pool)	36,265
Contracts used for paid-rate statistics (dataset C sample)	300,000 of 1,292,791
Persona hourly floors from real paid rates (ai_ml / fullstack / data_analyst)	$30 / $21 / $30
Dev-set jobs (3 personas × 200), GigSim live jobs	600, 180

Preprocessing: HTML unescape, replacing emails, phones and links with placeholders, de-duplication, an English and length filter, a keyword categoriser into 13 categories plus "other", and paid-rate statistics per category. **Labels** are *silver*: rules over the client's own Upwork tags and the skills the portfolio text can prove (a job is a fit when at least 60% of its tags are provable and an evidence project exists), plus the client's verified payment and a market-based lowball rule. A 45-row audit sheet is provided for a human check. **Limitations:** the labels and the Fit Matcher both reason from skill evidence (partly circular); A is from 2023; keyword categories are noisy; reply rates still come from a simulator I wrote.

3. Architecture

Two LangGraph `StateGraph`s, both rendered from code in `docs/images/`. The **bid graph** (one thread per job): Scout → Job Analyst, Risk Assessor and Fit Matcher in parallel (`Send`) → a deferred `decide` node (skip / watch / bid) → Pricing → Writer → claim verifier → Critic (loops back at most twice) → Compliance Guard → dry-run preview → `interrupt()` for approval → submit (or re-check an edit, or regenerate a paragraph). The **conversation graph** (one thread per client conversation): context → intent (question, negotiation, interview request, scam signal) → Client Liaison (reply + interview slots, walk-away floor) → Compliance → preview → approval → send.

Marketplace port. One `MarketplacePort` protocol with two adapters, GigSim and Upwork live, behind our own Marketplace MCP server. Agents only see MCP tools; the mode is configuration. **MCP surface:** three own servers (marketplace, portfolio, workspace), plus the official Upwork and GitHub servers; per-agent allowlists; write tools belong only to the post-approval executor (`docs/TOOLS.md`). **Stores:** Postgres for app data, checkpoints and the memory store (SQLite locally), an append-only audit log, Redis for the Arq queue and live events.

State, trace and memory are separate. State is the typed per-thread record (`agent/state.py`, reducers on every field parallel branches write). The trace is every node, model call and tool call with arguments, tokens, cost and latency (`trace_spans`, one trace id from the browser to the audit log). Long-term memory holds only promoted knowledge: a style rule after the same edit appears in two different drafts; an outcome insight after three outcomes with a clear reply-rate gap.

Why each agent exists is in `docs/ARCHITECTURE.md`. The short version: Risk runs before writing because scams are the costliest mistake; the Matcher builds a requirement-by-evidence matrix because "fit" without evidence is a guess; the verifier exists because writers overclaim (§5); the Critic loop is bounded because unbounded self-critique burns tokens without converging.

4. R&D question A: tool granularity

Hypothesis. Task-shaped tools (named after what an agent is trying to do) need fewer calls and fewer schema tokens than API-shaped fine-grained tools, and are easier to select than one mega-tool with an `action` argument.

Setup. The same capabilities exposed three ways (8 task-shaped, 22 fine-grained, 2 mega tools), 40 tasks (8 templates × 5 entities) with the correct first call(s) per design (`evals/runners/tool_granularity.py`).

Tool design	Tools	Schema tokens per turn (≈)	Min calls / task	Selection accuracy
task_shaped	8	479	1.12	not run
fine_grained	22	1111	2.25	not run
mega	2	166	1.12	not run

Result. Fine-grained tools cost 2.3× the schema tokens on *every* model turn and need twice the calls per task. The mega-tool is the cheapest in tokens, but its arguments are an untyped `params` object, so argument validation moves from the MCP schema into the model's head. Selection accuracy needs a tool-calling model and was not run here. **Conclusion:** task-shaped tools ship (ADR 0003). The open question for the LLM run is whether mega-tool argument accuracy falls far enough to cancel its token advantage.

5. R&D question B: advanced RAG

Hypothesis (stated before running): requirement decomposition plus reranking raises evidence Recall@5 over the naive pipeline and, through better-grounded drafts, raises the GigSim reply rate by at least 3 points.

Variants. V0 naive (fixed 512-token chunks, dense only), V1 requirement decomposition, V2 hybrid BM25 + dense (RRF), V3 reranking, V4 outcome boost (evidence from proposals that won), V5 multi-vector (summary / README / code / proposal indexed separately), V6 contextual chunk headers, V7 HyDE, V8 corrective (grade, re-query, refuse). Selection rule fixed in advance: best nDCG@5 on the tune split (2 dp), then MRR, then latency.

Variant (tune split)	Recall@5	MRR	nDCG@5	p95 ms
v0_naive	0.498	0.547	0.447	7.5
v1_decompose	0.513	0.505	0.430	4.7
v2_hybrid	0.503	0.552	0.456	8.4
v3_rerank	0.487	0.494	0.423	8.9
v4_outcome	0.518	0.521	0.444	6.3
v5_multivector	0.421	0.383	0.316	15.6
v6_contextual	0.511	0.566	0.471	6.2
v7_hyde	0.607	0.627	0.537	5.4
v8_corrective	0.559	0.560	0.480	5.2
v2_hybrid_v1_decompose	0.531	0.520	0.445	4.7
v3_rerank_v1_decompose	0.535	0.532	0.453	11.5
v2_hybrid_v1_decompose_v4_outcome	0.519	0.484	0.425	3.9
v2_hybrid_v1_decompose_v3_rerank_v4_outcome	0.541	0.496	0.442	11.4
v6_contextual_v2_hybrid_v1_decompose_v4_outcome	0.513	0.501	0.430	5.0
v8_corrective_v2_hybrid_v1_decompose_v4_outcome	0.543	0.529	0.461	6.6
v7_hyde_v2_hybrid	0.654	0.696	0.601	6.0
v7_hyde_v2_hybrid_v6_contextual	0.644	0.692	0.599	6.9

Chosen by rule (max nDCG@5 on tune split (2 dp), then MRR, then lower p95 latency): **v7_hyde_v2_hybrid**. Held-out test split: nDCG@5 0.582 vs naive 0.450; Recall@5 0.626 (95% CI [0.5645, 0.6901]) vs 0.516. Embedder `hash`, reranker `lexical`, commit `9ba9cd9`.

Ablation of the shipped pipeline (tune split), and the components the rule did not pick:

Setting (tune split)	Recall@5	nDCG@5	Δ nDCG@5
shipped (all components)	0.654	0.601	-
structured chunking → fixed 512-token chunks	0.605	0.539	-0.0625
requirement decomposition off	0.654	0.601	0.0000
reranking off	0.654	0.601	0.0000
+ hybrid BM25	0.654	0.601	0.0000
+ contextual chunk headers	0.645	0.600	-0.0011
+ outcome boost	0.694	0.573	-0.0279
+ HyDE	0.654	0.601	0.0000

On real posts the picture changed from the synthetic build, where requirement decomposition won. **HyDE** (retrieve with a hypothetical ideal proposal) helps most, because real posts describe a problem in the client's words, and a hypothetical proposal translates them into the portfolio's vocabulary. Hybrid BM25 on top adds another +0.06. With the shipped pipeline, **structured chunking is now the component that matters most** (−0.063 nDCG@5 with fixed 512-token chunks). Two negative results held from the synthetic build: **outcome boosting hurts ranking** (−0.028; it promotes past proposals that were won for a *different* requirement), and **multi-vector grouping (V5) is the worst variant**. Retrieval on real posts is much harder overall: V0 Recall@5 falls from 0.92 on synthetic jobs to 0.52. Real posts are vaguer, and the ai_ml persona is the hardest slice (the shipped pipeline is slightly *worse* than V0 on its Recall@5).

By persona and job type (test split; reply rate from the end-to-end run below):

Slice (V0 → shipped)	Recall@5	nDCG@10	GigSim reply rate
persona: ai_ml	0.428 → 0.392	0.359 → 0.413	
persona: fullstack	0.670 → 0.836	0.557 → 0.794	
persona: data_analyst	0.441 → 0.641	0.438 → 0.589	
job type: duplicate (n=2)			32.3% → 43.1%
job type: fit (n=32)			26.5% → 36.6%
job type: injection (n=6)			13.4% → 21.0%
job type: off_domain (n=2)			23.2% → 27.9%
job type: weak_fit (n=5)			36.5% → 40.8%

End to end (B5). Retrieval metrics alone do not decide what ships, so V0 and the shipped pipeline were run through the whole bid graph:

Variant	Evidence Recall@5	nDCG@10	Unsupported claims / proposal	Judge score	GigSim reply rate	p95 latency (s)	Cost / proposal (USD)
V0 naive	0.516	0.453	0.00 (0.30 before verify)	3.66	26.0%	0.39	$0.0000
Shipped: v7_hyde_v2_hybrid	0.626	0.602	0.00 (0.26 before verify)	4.00	34.9%	0.43	$0.0000

Paired on the same 47 proposals: the lead evidence changed in only 10, yet reply probability rose +8.2 points on the proposals whose lead evidence did *not* change. Drafts quoting a specific metric: 19.1% with V0 vs 46.8% shipped.

Claim verification. Without the verifier, the shipped writer leaves the unsupported-claim rates shown in the "before verify" column; after it, zero remain on the test split. The verifier treats honest negatives ("I have not used X in client work") as non-claims, otherwise it deleted the most trustworthy sentence in the draft.

Judge calibration. The 30-item blind labelling set is in `evals/datasets/judge_labels.csv`. Human labels cannot be produced by the system, so Cohen's κ is **pending** until the 30 labels are filled in; the only agreement reported is a clearly labelled proxy against the GigSim client model:

Judge calibration (30 items)	Value
Mean judge score: GigBid / reuse past / template	3.91 / 2.33 / 2.75
Agreement with human labels	awaiting human labels (0/30)
Proxy: quadratic κ vs GigSim client model (not human)	0.4179

Where retrieval and replies came apart. The stated hypothesis (decomposition + reranking) was wrong: neither was picked on real posts. The broader claim, that better retrieval raises replies by at least 3 points, held. But the paired comparison under the B5 table shows the same mechanism found on synthetic data. The lead evidence changed for only a minority of proposals, and most of the gain is on proposals that cite the *same* project. What changed is the chunk the Writer receives: fixed 512-token chunks cut case studies mid-way, so the "Result" section with the number often lands in another chunk. Structured chunks keep problem, approach and result together, and the share of drafts quoting a specific metric more than doubles. nDCG cannot see this. The flip side is that the writer attempts more claims as evidence improves, which only works because the verifier sits behind it.

Shipped: `v7_hyde_v2_hybrid` over structured chunks. It wins the declared rule on the tune split, holds on the test split, and costs a few milliseconds. Structured chunking is the part we would never remove.

6. R&D question C: is the multi-agent design worth it?

Same 50 real test jobs (17/17/16 per persona): (a) one ReAct agent with every read tool, (b) `langgraph-supervisor` with risk, matcher, pricing and writer workers, (c) the hand-written graph.

Architecture (50 jobs)	Decision accuracy	Scam bids	Unsupported claims (independent re-check)	Tool calls / job	p95 latency
graph	92.0%	0	6.2%	10.0	0.45s
react	not run: needs an LLM				
supervisor	not run: needs an LLM				

The “unsupported claims” column re-checks each final draft against an independent evidence search, which is stricter than the shipped verifier’s per-requirement evidence. Only the graph can run without a model; (a) and (b) are implemented (`evals/runners/architecture_compare.py`) and run with `make eval-llm`. What the graph guarantees structurally, whatever a model does: risk always runs before writing, unsupported claims are always removed, and no write tool is reachable before approval. A ReAct agent can only be *asked* to do those things. The comparison to watch in the LLM run is whether (a) wins on simple single-skill jobs, where the graph’s three parallel analyses are overhead.

7. The change request (2 Nov policy update)

What changed. Marketplace policy v2: bidding in a job’s first hour costs 2 extra Connects, and every screening question must be answered (the stand-in implements this as `GIGSIM_POLICY_VERSION=2`).

What had to be edited. Pricing stores the policy it already fetched through `get_policy` in state (3 lines); the Compliance Guard gained rule MP2, blocking drafts with an unanswered screening question (9 lines); one integration test (`test_policy_v2_change_request`). No adapter, graph, prompt or UI change: the Connects cost already came from the marketplace, and the approval screen already shows it. **Time:** about 20 minutes including the test.

What it says about the design. Policy is data behind a read tool, and enforcement lives in one node. The change would have been a multi-file edit if prices or Connects had been computed inside the Writer.

8. Signature extension: X2 proposal A/B experiments

Each bid is assigned an arm by a stable hash (user, experiment, job), so resumes never flip it; the Writer honours the arm; analytics run a two-proportion z-test and only declare a winner after 30 proposals per arm and $p < 0.05$ (`docs/EXTENSION.md`). The shipped experiment: ask the client one specific clarifying question, or close with a concrete next step.

X2 experiment: ask one clarifying question vs close with a next step	Value
Jobs (paired, both arms written)	47
Reply probability: statement / question	27.2% / 32.9%
Paired lift (question − statement)	+5.8 points (95% CI [4.3, 7.3])
Chance the live z-test calls it at 30 / 120 / 500 proposals per arm	10.0% / 20.0% / 47.5%

The paired design (both arms written for the same job) shows a clear lift; the simulated live experiment shows the uncomfortable part: at one freelancer’s volume, even 500 proposals per arm give less than an even chance of calling a lift of this size. The first version of this experiment (question-first vs evidence-first opening) measured a difference of exactly zero because both arms still contained a question; it was redesigned (`docs/AI_USAGE.md`).

9. Failure analysis

The big one: a system tuned on synthetic data. Run unchanged on the real-job dev set, the first build kept its safety record but bid on only 3 of the 50 test jobs it should have:

Test split (180 jobs each)	Bid/skip accuracy	Bid F1	Bid recall	Scams not bid on	Injections obeyed	Price in fair band
Synthetic jobs, old code	91.7%	0.928	0.924	100.0% of 17	0 of 19	84.6%
Real jobs, old code	73.3%	0.111	0.060	100.0% of 27	0 of 14	100.0%
Real jobs, after fixes	89.4%	0.800	0.760	100.0% of 27	0 of 14	88.9%

Evidence retrieval (V0 naive, test split): Recall@5 0.922 on synthetic jobs vs 0.516 on real jobs.

Every failed job was grouped into the taxonomy (`evals/runners/failure_analysis.py`). The fixes were diagnosed and chosen on the **tune** split only, and the test split was scored once at the end:

Failure type (real-job test split, 180 jobs)	Before (code tuned on synthetic jobs)	After fixes
wrong skip: risk/fit threshold	38	8
wrong skip: injected post treated as client risk	9	4
wrong bid: weak fit (client's skill tags mostly not in portfolio)	1	5
wrong bid: lowball budget	0	0
wrong bid: off-domain (missed evidence gap)	0	2
wrong bid: repost not recognised	0	0
wrong bid: scam	0	0
mispriced	0	5
unsupported claim	0	0
injection obeyed	0	0
tool error	0	0
bid/skip accuracy	73.3%	89.4%
bid F1 (precision / recall)	0.111 (0.750 / 0.060)	0.800 (0.844 / 0.760)
bids inside fair price band	100.0% of 4	88.9% of 45

1. **Umbrella tags (missed evidence → wrong skip).** Real clients tag "Machine Learning", "Data Science", "Web Development"; the portfolio says "LightGBM", "pandas", "Next.js". The matcher scored those requirements as unevidenced, and fit collapsed (0.07–0.20 on jobs the persona clearly fits). Fix: an umbrella-tag ontology built from the most common real tags per persona (`rag/index/skills.py: BROAD_TAGS`). Evidence for a child skill now covers the umbrella tag, and the tag is expanded in retrieval queries. **Worked:** fit on those jobs rose to 0.6–0.8.
2. **Made-up price floors.** Persona floors of $40–45/h came from my own guess. Real paid medians in the personas' categories are $21–30/h, so jobs a typical freelancer takes were skipped as "below floor". Fix: floors are now the median paid rate from 300k real contracts, and the fixed-price minimum went from 8 to 4 hours at the floor (real fixed contracts have a paid p25 of $50–90).
3. **Thresholds.** The fit and risk thresholds swept on synthetic data (0.70 / 0.35) were far too strict for real clients, whose posts are vaguer and who carry more risk signals (35% have unverified payment). Re-swept on the real tune split: 0.35 / 0.60.
4. **Scam rules that fired on ordinary work (§10).** On 5,000 real posts the v2 rules flagged 2.9% as scams, mostly Telegram and Discord bot builds and crypto products. v3 requires an *ask* ("contact me on Telegram", "pay in crypto", "send your phone number"), not just a word. The kit's sample "external messaging app" job exposed one more gap.
5. **Under-pricing.** Of the 32 mispriced tune-split bids, 24 were under-bids, for example $270 on a $4,000 job. The market band from real contracts is far below most posted budgets, because most contracts are small. Fix: with a budget, quote a fit-dependent share of it (80–95%); the market band is used only when there is no budget.

6. **Retrieval re-selected.** The declared rule picked HyDE on real posts; its ablation showed hybrid search adding +0.06 nDCG@5, so the combination was added as a candidate, and it wins the same rule (§5).

Not fixed. Hourly jobs whose maximum sits just under the floor are still bid at the floor, slightly above the client's range. Some weak-fit jobs pass, where the client's tags are mostly generic ("Web Development", "API"). Injected posts that also ask for contact details are still treated as client risk.

10. Safety and platform respect

Must never happen: a send without approval; Connects spent twice; an instruction inside a job post or client message being followed; contact details or off-platform payment in a proposal; scraping or browser automation of Upwork; one user's data reaching another.

How it is enforced. Write tools are reachable only by the executor after `interrupt()` returns approve (allowlists, tested); idempotency keys from thread + version + edits (crash-and-resume test, exported trace); job posts and messages are sanitised and delimited, and the Compliance Guard blocks or rewrites P1–P7 / MP2 violations after any human edit too; PII masking in logs and traces; OAuth tokens encrypted with Fernet; every query scoped by user id; append-only audit log; Upwork only through its official MCP server, polled at most every 4 hours.

Planted-injection jobs (test split)	Attack success
Defences off (no sanitising, no delimiting)	not measurable offline (the heuristic writer cannot follow instructions); run `make eval-llm`
Defences on (shipped)	0 of 14

Scam rules on real text. Synthetic scams were written by the same author as the rules, so their recall says little. Two real-text checks: the flag rate on 5,000 real posts (mostly legitimate, so flags are mostly false alarms), and an external fake-job benchmark (dataset F, scored on its held-out 30%):

Scam rules on real text	v2 (before)	v3 (shipped)
Scam-level flags on 5,000 real Upwork posts	2.9%	0.4%
External fake-job benchmark: recall (n=1200, 50% fraud)	0.0%	80.7%
External fake-job benchmark: precision	0.0%	99.8%

A TF-IDF + logistic-regression detector trained on the benchmark's own tune split reaches precision 99.5% / recall 93.4%; its top features (earn, no, immediate, hiring, knowledge in) show the fraud class is templated, so this is an upper bound for that dataset, not for Upwork.

The v2 rules caught **none** of the benchmark's frauds, which are "earn $5,000 a week, no experience, contact now" ads, a scam type the synthetic data never had. A new income-scam rule catches 81% at 99.8% precision, without raising the false-alarm rate on Upwork posts. The learned baseline is better on that dataset, but its fraud class is templated, and the data's licence is unknown, so no model trained on it ships.

What still gets through. Attack success *before* defences can only be measured with an LLM writer (the offline writer cannot follow instructions by construction), so the "after" zero is necessary but not sufficient evidence. A client can still move a conversation off-platform in their *own* words; GigBid flags it as a scam signal but the human decides. Paraphrased injections that avoid the regex layer reach the model inside delimiters; only the LLM run tells how often the model obeys them.

11. Results against the declared budgets

The hidden test set is run by the graders. On our held-out test split:

Metric (test split, n=180, profile `offline`)	Value
Bid/skip accuracy	89.4% (95% CI [84.4, 93.3])
Bid precision / recall / F1	0.844 / 0.760 / 0.800
Scam posts not bid on	100.0% of 27
Injected posts flagged / obeyed	100.0% flagged, 0 of 14 obeyed
Unsupported claims without verifier → shipped	11.8% → 0.0% (102 claims)
Bids inside the fair price band	88.9% of 45
Latency per job p50 / p95	0.26s / 0.45s
Errors	0 of 180

Thresholds chosen on the tune split by F1: bid_min_fit=0.35, max_risk=0.6 (tune F1 0.8308).

System (180 test jobs)	Bids	Connects	Reply rate	Replies / 100 Connects	Interviews	Hires	Scam bids
GigBid	47	257	36.0%	6.58	10.8	5.4	0
Template, bid on all	115	587	11.2%	2.20	6.7	2.7	5
Reuse best past proposal	180	861	11.7%	2.44	11.2	4.7	27

Measured latency is far inside the 25 s budget, and cost is zero, **because no model was called**; these numbers say nothing about the open profile, whose p95 and cost come from `make eval-llm`. Against the baselines on the same jobs, GigBid bids on fewer jobs, spends fewer Connects and gets more replies per Connect, and never bids on a scam; the caveat is circularity: the stand-in simulator rewards what its author believed clients reward. One optimisation that helped: skipping early (scam and fit checks before any writing) means most jobs never reach the Writer.

Appendix A. Reproduce

```
make install && make data && make test && make e2e && make eval && make report
```

Commits, traces and results: `git log`, `docs/traces/*.json`, `evals/results/*.json`.